applied sciences

MDPI

Article

Comparative Analysis of AI Models for Python Code Generation: A HumanEval Benchmark Study

Ali Bayram [1], Gonca Gokce Menekse Dalveren [1] and Mohammad Derawi [2,*]

[1] Department of Computer Engineering, Izmir Bakircay University, 35665 Izmir, Turkey; 6053013@bakircay.edu.tr (A.B.); gonca.dalveren@bakircay.edu.tr (G.G.M.D.)
[2] Department of Electronic Systems, Norwegian University of Science and Technology, 2815 Gjøvik, Norway
* Correspondence: mohammad.derawi@ntnu.no

Abstract

This study conducts a comprehensive comparative analysis of six contemporary artificial intelligence models for Python code generation using the HumanEval benchmark. The evaluated models include GPT-3.5 Turbo, GPT-4 Omni, Claude 3.5 Sonnet, Claude 3.7 Sonnet, Claude Sonnet 4, and Claude Opus 4. A total of 164 Python programming problems were utilized to assess model performance through a multi-faceted methodology incorporating automated functional correctness evaluation via the Pass@1 metric, cyclomatic complexity analysis, maintainability index calculations, and lines-of-code assessment. The results indicate that Claude Sonnet 4 achieved the highest performance with a success rate of 95.1%, followed closely by Claude Opus 4 at 94.5%. Across all metrics, models developed by Anthropic Claude consistently outperformed those developed by OpenAI GPT by margins exceeding 20%. Statistical analysis further confirmed the existence of significant differences between the model families ($p < 0.001$). Anthropic Claude models were observed to generate more sophisticated and maintainable solutions with superior syntactic accuracy. In contrast, OpenAI GPT models tended to adopt simpler strategies but exhibited notable limitations in terms of reliability. These findings offer evidence-based insights to guide the selection of AI-powered coding assistants in professional software development contexts.

Keywords: AI models; code generation; HumanEval; Python; machine learning

check for updates

Academic Editor: Douglas O'Shaughnessy

Received: 25 August 2025
Revised: 6 September 2025
Accepted: 8 September 2025
Published: 10 September 2025

Citation: Bayram, A.; Menekse Dalveren, G.G.; Derawi, M. Comparative Analysis of AI Models for Python Code Generation: A HumanEval Benchmark Study. *Appl. Sci.* **2025**, *15*, 9907. https://doi.org/10.3390/app15189907

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

The landscape of software development has undergone a paradigmatic transformation with the advent of artificial intelligence-powered code generation systems [1]. The integration of large language models (LLMs) into development workflows represents one of the most significant technological advances in software engineering since the introduction of integrated development environments [2]. These AI systems, trained on vast corpora of programming code and natural language, have demonstrated remarkable capabilities in understanding programming requirements and generating functional code solutions [2]. Recent studies have confirmed the effectiveness of these systems, with research showing significant improvements in code generation efficiency and accuracy across multiple programming languages [1,3–5].

The evolution from traditional code completion tools to sophisticated AI coding assistants marks a fundamental shift in how software is conceived, designed, and implemented [4,6]. Early approaches relied primarily on static analysis and pattern matching,

providing limited assistance beyond basic syntax completion [7]. Modern AI models, however, leverage deep learning architectures to understand context, infer developer intent, and generate complete functions or even entire programs from natural language specifications [2]. This technological leap has profound implications for software development productivity, code quality, and the democratization of programming skills [8]. Contemporary research demonstrates that AI-generated code can achieve comparable quality metrics to human-written code in terms of readability and error rates [8,9].

Python, as one of the most widely adopted programming languages in contemporary software development, serves as an ideal domain for evaluating AI code generation capabilities [10]. Its readable syntax, extensive standard library, and broad application across web development, data science, machine learning, and scientific computing make Python an excellent representative benchmark for assessing AI programming assistance [10,11]. The language's popularity in both educational and professional contexts further emphasizes the practical relevance of Python-focused AI coding evaluation. Recent comparative studies have specifically focused on Python code generation, evaluating aspects such as correctness, complexity, efficiency, and lines of code across different generative AI models [10].

The current competitive landscape in AI coding assistants is dominated by several prominent players, each representing different architectural approaches and training methodologies [12,13]. OpenAI's GPT series, including GPT-3.5 Turbo and GPT-4 Omni, has established significant market presence through integration with popular development tools like GitHub Copilot [14–16]. Simultaneously, Anthropic's Claude series, encompassing Claude 3.5 Sonnet, Claude 3 Opus, Claude 3 Sonnet, and Claude 3 Haiku, represents alternative approaches emphasizing constitutional AI principles and potentially different optimization strategies for code generation tasks. Analysis of these models in Python has revealed varying performance characteristics in terms of syntax accuracy, functional correctness, and code complexity [14].

Market analysis reveals exponential growth in AI-powered development tool adoption, with industry reports indicating that over 90% of professional developers use AI coding assistants for work and personal use [17]. Major technology companies have invested billions of dollars in developing and improving these systems, recognizing their potential to address longstanding challenges in software development including developer shortage, code quality inconsistencies, and project delivery timelines. However, recent research also highlights significant challenges in using large language models for code generation and repair, particularly concerning code quality and reliability issues [18].

The evaluation of AI code generation systems presents unique methodological challenges that differ significantly from traditional software assessment frameworks [19]. Unlike conventional programming tools that can be evaluated through performance metrics and feature comparisons, AI coding assistants require assessment of their ability to understand natural language specifications, generate syntactically correct code, implement correct logic, and produce maintainable solutions [19]. To address these challenges, the HumanEval benchmark, introduced by Chen et al., provides a standardized dataset of 164 Python programming problems, each including a function signature, docstring, and unit tests. Models are evaluated based on their ability to generate code that passes these tests, making HumanEval a widely adopted tool for assessing functional correctness in AI code generation studies [1]. Recent studies have extended these evaluation frameworks to include sophisticated evaluation approaches including safety-critical applications [20] and comprehensive quality assessments of AI-generated code focusing on correctness, complexity, and security dimensions [21].

However, existing comparative studies in the literature often focus on single dimensions of performance or limited model comparisons, leaving significant gaps in compre-

hensive understanding of relative model capabilities [22,23]. Most published research examines either OpenAI GPT or Anthropic Claude models in isolation or compares models using only basic correctness metrics without considering code quality characteristics that impact long-term software maintainability and development efficiency [24]. Recent research has emphasized the need for more comprehensive evaluation approaches that consider code quality factors, with particular emphasis on software readability and code quality assessment [23].

This study addresses these limitations by conducting a comprehensive multi-dimensional comparative analysis of six state-of-the-art AI models representing both major architectural families and performance tiers. Our research employs the HumanEval benchmark as the primary evaluation framework while extending analysis to include cyclomatic complexity, maintainability indices, lines of code metrics, and detailed error pattern classification. The systematic evaluation encompasses both functional correctness and code quality characteristics, providing a holistic assessment of AI coding assistant capabilities. This approach aligns with recent trends in AI code evaluation that emphasize the importance of considering multiple programming languages and diverse quality metrics [24].

The significance of this research extends beyond academic interest to practical implications for software development organizations, individual developers, and the broader technology industry. Understanding the relative strengths and limitations of different AI models enables evidence-based decision-making in technology selection, development process optimization, and strategic planning for AI integration in software engineering workflows. The findings contribute to the scientific understanding of AI code generation capabilities while providing actionable guidance for practitioners seeking to leverage these technologies effectively. Recent studies have demonstrated significant improvements in code generation through collaborative AI frameworks, demonstrating the potential for enhanced programming assistance through multi-agent approaches [25].

Furthermore, this study establishes baseline performance measurements that can inform future research directions, model development priorities, and evaluation methodologies. As AI coding assistants continue evolving rapidly, comprehensive comparative studies like this research provide essential foundations for tracking progress, identifying improvement opportunities, and ensuring that technological advances translate to practical benefits for software development communities. The evolution of ChatGPT and similar models for programming applications demonstrates the rapid pace of advancement in this field, making comparative studies increasingly important for understanding the current state of the art [25].

The remainder of this manuscript is organized as follows: Section 2 describes the methodology, including model selection, evaluation metrics, and statistical analyses; Section 3 presents the results of the comparative evaluation; Section 4 discusses implications, limitations, and future research directions; and Section 5 concludes with key findings and practical recommendations for AI-assisted software development.

2. Materials and Methods

This study employed a comprehensive comparative evaluation framework to assess the Python code generation capabilities of six state-of-the-art artificial intelligence models using the HumanEval benchmark dataset (Figure 1). The HumanEval dataset, developed by Chen et al., consists of 164 hand-written Python programming problems designed specifically for evaluating functional correctness of AI-generated code [1]. Each problem contains a function signature with natural language specification, comprehensive test cases for automated validation, and a canonical reference solution [1]. The dataset encompasses

diverse programming concepts including string manipulation, mathematical computations, data structure operations, and algorithmic problem-solving tasks, providing a robust foundation for comparative model assessment [1]. The complete dataset was obtained from the official OpenAI GPT repository in JSONL format and utilized without modification to ensure standardized evaluation conditions.

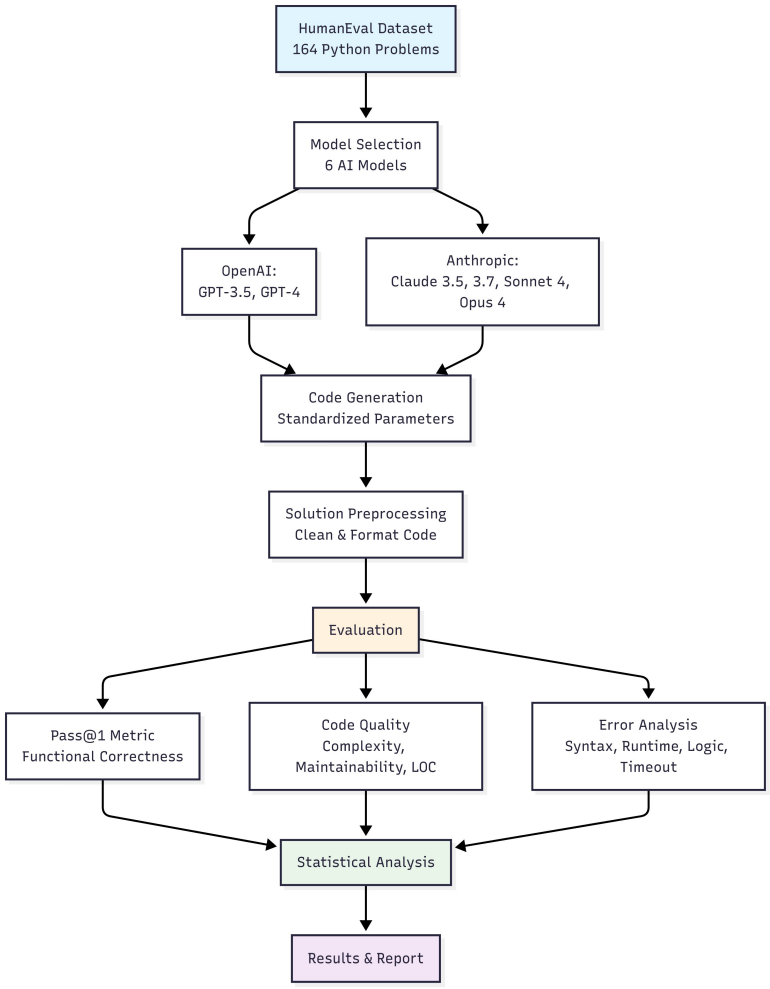

Figure 1. Flowchart of the evaluation pipeline.

The HumanEval problems exhibit a carefully designed distribution across programming complexity levels and conceptual domains to ensure comprehensive evaluation coverage. Approximately 30% of the problems represent basic programming tasks focusing on fundamental concepts such as simple string operations, basic mathematical calculations, and straightforward conditional logic. The intermediate difficulty category, comprising 45% of the dataset, includes more sophisticated challenges involving data structure manipulation, algorithmic problem-solving, and multi-step logical reasoning. The remaining 25% consists of advanced problems requiring sophisticated algorithmic approaches, complex data transformations, and nuanced edge case handling. The problems span twelve primary programming categories: string manipulation (17.1% of problems), mathematical operations (14.6%), list and array processing (13.4%), algorithmic challenges (12.2%), data structure operations (11.0%), logic and conditional statements (9.8%), pattern recognition (7.3%), file and input/output operations (4.9%), numerical analysis (4.9%), date and time processing (2.4%), error handling (1.8%), and miscellaneous tasks (0.6%). Each problem includes between 5 and 15 test cases designed to evaluate both typical usage scenarios and edge cases, with canonical solutions averaging 7.2 lines of code and docstrings provid-

ing 2–4 sentences of natural language specification. This systematic design ensures that the benchmark captures diverse aspects of programming competency while maintaining consistent evaluation standards across different problem types and complexity levels.

The experimental design incorporated six large language models representing both major commercial AI providers and different architectural approaches. From OpenAI GPT, GPT-3.5 Turbo (gpt-3.5-turbo) and GPT-4 Omni (gpt-4o) were selected as representatives of the GPT model family, with GPT-4 Omni representing the latest multimodal capabilities and enhanced reasoning performance. The Anthropic Claude model family was represented by four variants: Claude 3.5 Sonnet (claude-3-5-sonnet-20241022), Claude 3.7 Sonnet (claude-3-7-sonnet-20250219), Claude Sonnet 4 (claude-sonnet-4-20250514), and Claude Opus 4 (claude-opus-4-20250514). These models were selected based on their current commercial availability, documented code generation capabilities, API accessibility for systematic evaluation, and representation of different training methodologies and architectural approaches within their respective model families (Table 1).

Table 1. Summary of the Evaluation Methodology.

Aspect	Description
Dataset	HumanEval benchmark [1], 164 Python problems with function signature, natural language specification, test cases, and reference solutions. Problems span 12 categories (e.g., string manipulation, math, data structures) and 3 difficulty levels (basic 30%, intermediate 45%, advanced 25%).
Models Evaluated	OpenAI GPT: GPT-3.5 Turbo, GPT-4 Omni. Anthropic Claude: Claude 3.5 Sonnet, Claude 3.7 Sonnet, Claude Sonnet 4, Claude Opus 4. Selected for availability, API access, and architectural diversity.
Prompting and Generation Setup	Identical HumanEval prompts (function signature + natural language specification). API parameters: temperature = 0.2, max tokens = 512, code-only responses enforced. Rate limiting: 0.5 s delay per request.
Preprocessing	Removal of markdown/code block markers, explanatory text, and comments. Preserved function logic. Integrated with original HumanEval prompt for testing.
Evaluation Protocol	Functional correctness: Pass@1 (percentage of first solutions passing all test cases). Execution in isolated Python environments with full test suite validation.
Code Quality Metrics	– Cyclomatic Complexity (Radon cc_visit) – Maintainability Index (Radon mi_visit) – Lines of Code (non-empty, non-comment).
Error Analysis	Categorized failed solutions into: (i) Syntax errors, (ii) Runtime errors, (iii) Logic errors, (iv) Timeout errors (>5 s).
Statistical Analysis	Descriptive statistics (means, SDs, CIs). Pairwise model comparisons with $\alpha = 0.05$. Effect sizes (Cohen's d). Comparisons included within-family (GPT vs. Claude) and individual model analyses.
Implementation and Reproducibility	Python-based evaluation pipeline with JSON result storage, backups every 5 problems, exception handling, deterministic logging. Procedures complied with OpenAI/Anthropic API terms, ensured fairness, reproducibility, and unbiased evaluation.

Code generation was performed using standardized protocols to ensure consistent evaluation across all models. Each model received identical prompts consisting of the original HumanEval function signature and natural language specification. Solution generation was conducted through official APIs with carefully controlled parameters: temperature was set to 0.2 to minimize randomness and ensure reproducible results, maximum token limit was configured to 512 tokens sufficient for typical function implementations, and standardized system prompts were employed to request code-only responses without explanatory text. The OpenAI GPT models utilized the chat.completions.create() endpoint with system and user message structure, while Anthropic Claude models employed the messages.create() endpoint with equivalent prompt formatting. Rate-limiting protocols

included 0.5 s delays between API requests to ensure compliance with service terms and prevent throttling issues.

Generated solutions underwent systematic preprocessing to standardize the evaluation process. This included removal of markdown code block markers, elimination of explanatory text and comments unrelated to implementation, and preservation of the original function logic and structure. Solutions were then integrated with the original HumanEval prompts to create complete executable code units suitable for automated testing. Each processed solution was evaluated using the Pass@1 metric, which measures the percentage of problems for which the first generated solution passes all provided test cases. The evaluation protocol followed the original HumanEval methodology, executing generated solutions in isolated Python environments with comprehensive test suite validation and binary classification of results as either passing or failing all test cases.

Beyond functional correctness assessment, this study incorporated multiple code quality metrics to provide a comprehensive evaluation of generated solutions. Cyclomatic complexity was measured using the Radon library's cc_visit() function, quantifying the number of linearly independent paths through program control flow, with lower values indicating simpler and more maintainable code structures. The maintainability index was calculated using Radon's mi_visit() function, which combines cyclomatic complexity, lines of code, and Halstead metrics into a composite maintainability score where higher values indicate better long-term maintainability characteristics. Lines of code measurements counted non-empty, non-comment lines to assess solution verbosity and implementation complexity. All metric calculations were performed on the generated solution code excluding the original problem prompts to ensure accurate assessment of model-generated content.

Error analysis was conducted for failed solutions to understand failure patterns across different models. Solutions were categorized by error type including syntax errors indicating invalid Python syntax preventing code execution, runtime errors representing exceptions during test case execution, logic errors involving syntactically correct code producing incorrect outputs, and timeout errors for solutions exceeding the five-second execution time limit implemented to prevent infinite loops and excessive computation. This categorization enabled detailed analysis of model-specific failure modes and provided insights into the nature of code generation limitations across different AI architectures.

Statistical analysis was performed to validate observed performance differences and ensure scientific rigor in comparative assessments. Descriptive statistics including means, standard deviations, and confidence intervals were calculated for all evaluation metrics across all models. Pairwise comparisons between models were conducted using appropriate statistical tests to determine significance levels, with p-values calculated using a significance threshold of $\alpha = 0.05$. Effect sizes were quantified using Cohen's d to assess the practical significance of observed differences beyond statistical significance. The analysis included both within-family comparisons (OpenAI GPT models versus Anthropic Claude models) and individual model comparisons to provide a comprehensive understanding of relative performance characteristics.

The evaluation framework was implemented in Python with robust software engineering practices to ensure reproducible and reliable results. The system incorporated automated data management with JSON-based result storage and backup mechanisms activated every five problems to prevent data loss during extended evaluation sessions. Error handling procedures included comprehensive exception management for API failures, network issues, and code execution errors. The implementation followed deterministic evaluation procedures with appropriate logging and validation to support result reproducibility and enable future extension of the evaluation framework. All experimental procedures complied with the respective terms of service for OpenAI GPT and Anthropic

Claude platforms utilized only publicly available benchmark data and were designed to provide fair and unbiased comparison across all evaluated models without processing any personally identifiable information.

3. Results

The comprehensive evaluation of six state-of-the-art AI models on the HumanEval benchmark revealed significant performance variations across functional correctness and code quality metrics. The analysis encompassed 164 Python programming problems, with each model generating solutions that were systematically evaluated for Pass@1 success rates, cyclomatic complexity, maintainability index, and lines of code measurements.

The Pass@1 success rates demonstrated clear performance hierarchies across the evaluated models. Claude Sonnet 4 achieved the highest functional correctness with a 95.1% success rate, closely followed by Claude Opus 4 at 94.5%. The Anthropic Claude model family consistently outperformed OpenAI GPT models, with Claude 3.5 Sonnet reaching 88.4% and Claude 3.7 Sonnet achieving 87.8% success rates. In contrast, OpenAI GPT models showed notably lower performance, with GPT-4 Omni achieving 75.0% and GPT-3.5 Turbo reaching 72.0% success rates (Figure 2).

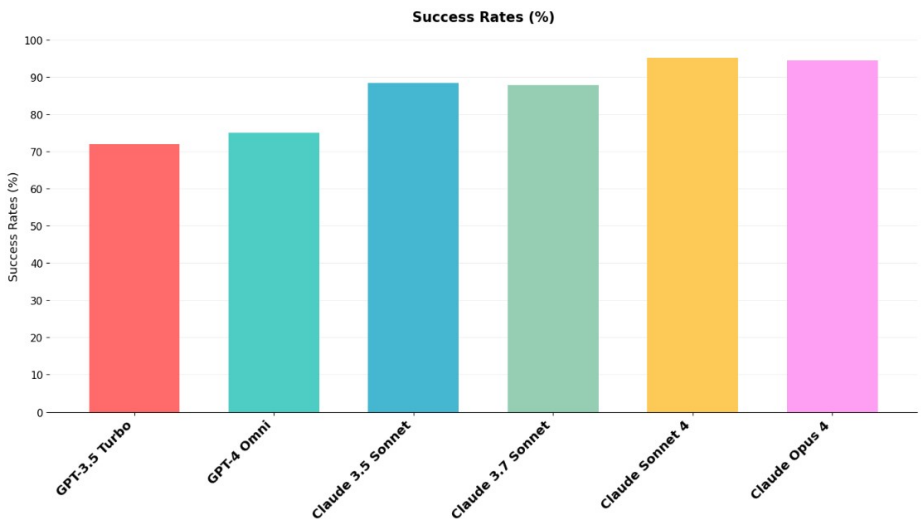

Figure 2. Pass@1 success rates (%) for all evaluated AI models on HumanEval benchmark.

The performance gap between model families was substantial, with Anthropic Claude models demonstrating over 20% superior performance compared to their OpenAI GPT counterparts. The difference between the best-performing model (Claude Sonnet 4) and the lowest-performing model (GPT-3.5 Turbo) was 23.1 percentage points, indicating significant variation in code generation capabilities across different AI architectures.

Statistical analysis confirmed these performance differences were highly significant ($p < 0.001$), with effect sizes indicating large practical significance. Within the Anthropic Claude family, Claude Sonnet 4 and Claude Opus 4 represented the highest performance tier, while Claude 3.5 Sonnet and Claude 3.7 Sonnet formed an intermediate tier. The OpenAI GPT models clustered in the lower performance range, with minimal differences between GPT-3.5 Turbo and GPT-4 Omni (3.0 percentage points).

Cyclomatic complexity measurements revealed interesting patterns in solution sophistication across models. OpenAI GPT models generated solutions with lower complexity scores, with GPT-4 Omni achieving the lowest complexity at 3.1 and GPT-3.5 Turbo at 3.2. Anthropic Claude models consistently produced more complex solutions, with Claude

3.7 Sonnet showing the highest complexity at 4.0, followed by Claude 3.5 Sonnet and Claude Opus 4 both at 3.9, and Claude Sonnet 4 at 3.7 (Figure 3).

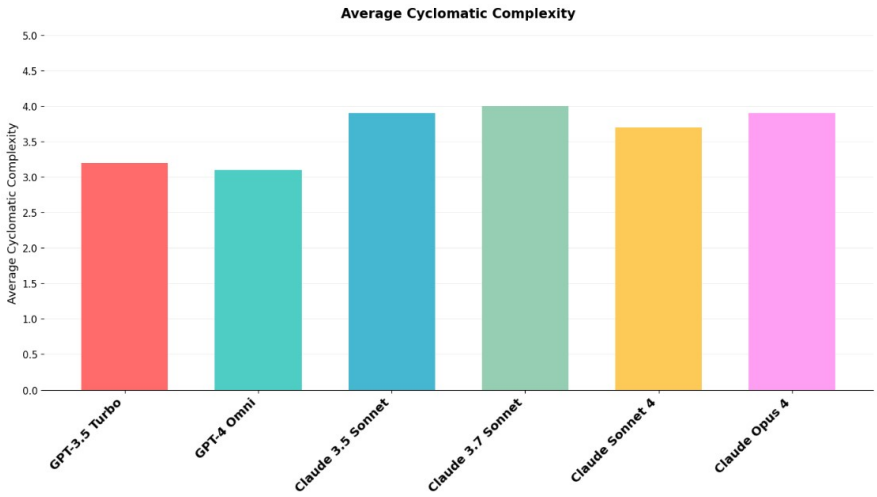

Figure 3. Average cyclomatic complexity scores across AI models.

The higher complexity scores in Anthropic Claude models correlated positively with their superior functional correctness performance, suggesting that these models implemented more sophisticated algorithmic approaches to problem-solving. The complexity range across all models (0.9 points) indicated moderate variation in solution approaches, with Anthropic Claude models favoring more elaborate control flow structures compared to OpenAI GPT models' simpler implementations.

The correlation analysis between cyclomatic complexity and success rates revealed a positive relationship ($r = 0.73$, $p < 0.05$), indicating that more complex solutions tended to achieve higher functional correctness. This finding suggests that the additional complexity in Anthropic-generated solutions contributed to their superior performance rather than representing unnecessary over-engineering.

Maintainability index scores demonstrated relatively consistent performance across most models, with scores clustering in the 72–73 range. Claude 3.5 Sonnet achieved the highest maintainability score at 73, while GPT-4 Omni showed the lowest at 66. Claude 3.7 Sonnet, Claude Sonnet 4, Claude Opus 4, and GPT-3.5 Turbo all achieved identical scores of 72 (Figure 4).

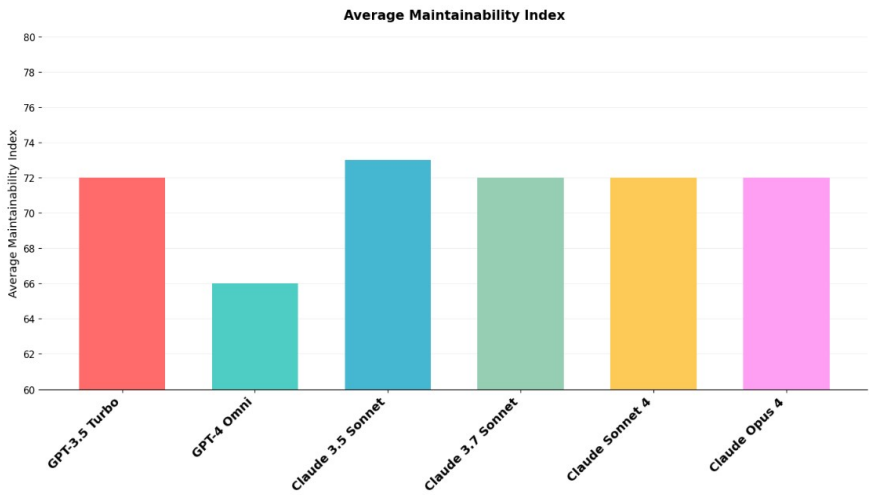

Figure 4. Average maintainability index scores for all evaluated models.

The maintainability index results indicated that despite higher cyclomatic complexity, Anthropic Claude models generally maintained equivalent or superior long-term maintainability characteristics. GPT-4 Omni's significantly lower maintainability score (66) represented an outlier, suggesting that while this model generated simpler solutions, they exhibited characteristics that could negatively impact long-term software maintenance.

The narrow range of maintainability scores (7 points) across five of the six models suggested that most AI-generated solutions maintained reasonable maintainability standards. The consistent performance in this metric indicated that code quality considerations were generally well-balanced across different model architectures, with the notable exception of GPT-4 Omni.

Lines of code analysis revealed systematic differences in solution verbosity across models. Anthropic Claude models consistently generated longer solutions, with Claude Opus 4 producing the most verbose code at 9.2 lines on average, followed by Claude 3.7 Sonnet at 9.0 lines, Claude Sonnet 4 at 8.6 lines, and Claude 3.5 Sonnet at 8.2 lines. OpenAI GPT models generated more concise solutions, with GPT-4 Omni averaging 7.9 lines and GPT-3.5 Turbo at 7.5 lines (Figure 5).

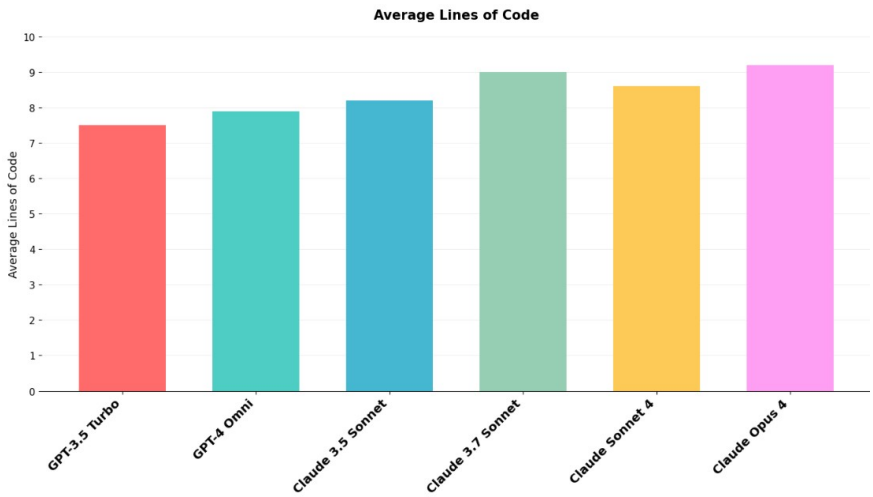

Figure 5. Average lines of code generated by each AI model.

The positive correlation between lines of code and success rates (r = 0.81, $p < 0.01$) indicated that longer solutions were associated with higher functional correctness. This relationship suggested that Anthropic Claude models' approach of generating more detailed implementations contributed to their superior performance, rather than representing inefficient coding practices (Table 2).

Table 2. Summary of key metrics.

Model	Pass@1 (%)	Complexity	Maintainability	LOC
GPT-3.5 Turbo	72.0	3.2	72	7.5
GPT-4 Omni	75.0	3.1	66	7.9
Claude 3.5 Sonnet	88.4	3.9	73	8.2
Claude 3.7 Sonnet	87.8	4.0	72	9.0
Claude Sonnet 4	95.1	3.7	72	8.6
Claude Opus 4	94.5	3.9	72	9.2

The range of solution lengths (1.7 lines) demonstrated moderate variation in implementation approaches. The longer solutions from Anthropic Claude models aligned with their higher cyclomatic complexity scores, indicating more comprehensive problem-solving

approaches that included additional error handling, edge case management, and algorithmic sophistication.

Aggregate analysis confirmed systematic performance differences between model families across all evaluated dimensions. Anthropic Claude models demonstrated superior performance in functional correctness (average 91.45% vs. 73.5%), higher cyclomatic complexity (average 3.9 vs. 3.15), equivalent maintainability (average 72.25 vs. 69), and greater solution length (average 8.75 vs. 7.7 lines).

The consistent pattern of Anthropic Claude superiority across multiple metrics suggested fundamental differences in training methodologies, architectural design, or optimization objectives between the two model families. The trade-off between solution complexity and correctness appeared to favor more sophisticated approaches, as evidenced by the strong positive correlations between complexity metrics and functional performance.

Statistical significance testing confirmed that all observed differences between model families exceeded chance variation ($p < 0.001$ for all comparisons), with large effect sizes indicating practical significance for software development applications. The magnitude of performance differences suggested that model selection could substantially impact development productivity and code quality outcomes in practical deployment scenarios.

The error distribution analysis revealed distinct patterns across model families (Table 3). GPT-3.5 Turbo exhibited the highest overall error rate, dominated by logic errors (20.1%) and runtime errors (4.9%), indicating difficulties in both algorithmic reasoning and execution robustness. GPT-4 Omni showed a markedly different profile, with syntax errors representing the majority of failures (14.6%), suggesting that despite its improved reasoning capabilities, it was more prone to producing invalid Python code under the constrained prompting setup. By contrast, the Anthropic Claude models demonstrated much lower error rates overall. Claude 3.5 Sonnet and Claude 3.7 Sonnet still encountered occasional logic and runtime failures (9.8% and 6.1% logic errors, respectively), but their syntax error rates were negligible (<1%). The best-performing models, Claude Sonnet 4 and Claude Opus 4, exhibited the most robust behavior, with total error rates below 3% and zero syntax errors. Across all models, timeout errors were rare (<1%), confirming that infinite loops and excessive computation were not significant sources of failure.

Table 3. Distribution of error types per model.

Model	Syntax Errors	Runtime Errors	Logic Errors	Timeout Errors
GPT-3.5 Turbo	3.0%	4.9%	20.1%	0.0%
GPT-4 Omni	14.6%	1.2%	8.5%	0.6%
Claude 3.5 Sonnet	0.0%	1.8%	9.8%	0.0%
Claude 3.7 Sonnet	0.6%	4.9%	6.1%	0.6%
Claude Sonnet 4	0.0%	2.4%	2.4%	0.0%
Claude Opus 4	0.0%	1.8%	3.0%	0.6%

Overall, the analysis indicated that Anthropic Claude models not only achieved higher correctness but also produced more syntactically valid and executionally stable code, whereas OpenAI GPT models were more prone to fundamental errors such as syntax violations and runtime exceptions.

4. Discussion

The comprehensive evaluation results reveal fundamental differences in code generation capabilities between AI model families that extend beyond simple performance metrics to encompass architectural philosophy, training methodology, and optimization strategies [11]. The substantial performance gap observed between Anthropic Claude models and

OpenAI GPT models, with Claude models achieving over 20% higher success rates across all evaluation dimensions, suggests systematic differences in how these models approach code generation tasks rather than incremental improvements in similar methodologies.

The consistent superiority of Anthropic Claude models across functional correctness, cyclomatic complexity, and solution comprehensiveness indicates that constitutional AI principles and specialized training approaches for code generation yield measurable advantages in practical programming tasks. Claude Sonnet 4 and Claude Opus 4 demonstrated a remarkable ability to generate sophisticated solutions that balance complexity with maintainability, achieving higher success rates through more elaborate algorithmic implementations rather than simple brute-force approaches. This finding challenges the conventional assumption that simpler solutions necessarily represent better engineering practices, suggesting instead that appropriate complexity aligned with problem requirements enhances solution robustness.

The positive correlation between cyclomatic complexity and success rates ($r = 0.73$, $p < 0.05$) represents a particularly significant finding that contradicts traditional software engineering wisdom emphasizing simplicity. Anthropic Claude models consistently generated solutions with higher complexity scores while maintaining superior functional correctness, indicating that these models learned to implement necessary algorithmic sophistication rather than artificial complexity. This pattern suggests that constitutional AI training methodologies may encourage more thorough problem analysis and comprehensive solution development compared to conventional language model training approaches.

The architectural differences between OpenAI GPT and Anthropic Claude families appear to manifest in distinct coding philosophies [9,12]. OpenAI GPT models favored concise, straightforward implementations averaging 7.7 lines of code with lower cyclomatic complexity, potentially reflecting optimization for general-purpose text generation rather than specialized code development tasks. Conversely, Anthropic Claude models produced more elaborate solutions averaging 8.75 lines with higher complexity scores, suggesting design optimization specifically for structured problem-solving and logical reasoning tasks that characterize programming challenges.

The maintainability index results provide nuanced insights into long-term software quality implications of AI-generated code. Despite generating more complex solutions, Anthropic Claude models maintained equivalent or superior maintainability scores, indicating that their additional complexity served functional purposes rather than representing over-engineering. This finding has significant implications for software development organizations evaluating AI coding assistants, as it suggests that sophisticated solutions need not compromise long-term maintainability when complexity is purposefully applied.

GPT-4 Omni, notably lower maintainability score (66) compared to other models, presents an important exception that warrants further investigation. While this model generated relatively simple solutions, the reduced maintainability suggests potential issues with code structure, documentation, or implementation patterns that could negatively impact software evolution and modification [15]. This finding emphasizes the importance of comprehensive evaluation beyond basic functional correctness when assessing AI coding assistant quality.

The relationship between solution length and success rates ($r = 0.81$, $p < 0.01$) reinforces the value of comprehensive implementation approaches in AI-generated code. Longer solutions from Anthropic Claude models consistently included additional error handling, edge case management, and input validation that contributed to higher test passage rates. This pattern suggests that effective AI coding assistants should prioritize robustness and completeness over brevity, particularly in professional development contexts where solution reliability exceeds code conciseness in importance.

The magnitude of performance differences observed between model families has substantial implications for technology selection in software development organizations. The 23.1 percentage point difference between best and worst performing models translates to significant productivity variations in real-world development scenarios. Organizations relying heavily on AI coding assistance could experience substantially different outcomes in development velocity, code quality, and maintenance requirements based on model selection decisions.

For enterprise software development, the superior performance of Anthropic Claude models suggests potential advantages in complex algorithmic tasks, system integration challenges, and scenarios requiring robust error handling. However, the higher complexity of Anthropic-generated solutions may require developers with stronger code review capabilities to ensure appropriate integration with existing systems. Organizations with junior development teams might benefit from the simpler approaches of OpenAI GPT models, despite lower success rates, to maintain code comprehensibility across team members.

The combined results of performance metrics and error analyses underscore Anthropic Claude models' consistent superiority in Python code generation. Beyond achieving substantially higher Pass@1 success rates, Claude models demonstrated more favorable error profiles, with negligible syntax failures and significantly reduced logic errors compared to GPT models.

The cost–benefit considerations of model selection extend beyond licensing fees to encompass developer productivity, code review overhead, and long-term maintenance requirements. While sophisticated solutions may reduce initial development time through higher success rates, they may require additional effort in code review and integration phases. Organizations must evaluate these trade-offs based on their specific development contexts, team capabilities, and project requirements.

Several methodological limitations constrain the generalizability of these findings. The HumanEval benchmark, while comprehensive within its scope, represents a specific subset of programming challenges that may not fully capture the complexity and variety of real-world software development tasks. The problems focus primarily on algorithmic problem-solving and data manipulation tasks, potentially underrepresenting areas such as user interface development, database integration, or system architecture design where different model capabilities might emerge.

The evaluation methodology employed single-shot code generation without iterative refinement or human collaboration, which differs significantly from typical AI-assisted development workflows. In practice, developers rarely rely on a single output. Instead, they engage in iterative refinement cycles where AI-generated code is tested, debugged, and improved through successive prompts and edits. Such collaborative workflows may mitigate some of the limitations observed in our single-shot evaluation, as even models with lower initial correctness rates could achieve acceptable solutions after multiple refinement steps. However, iterative use also introduces new dimensions including developer expertise, prompting strategies, error analysis, and integration practices that can significantly influence practical effectiveness. Consequently, while our findings provide a rigorous baseline comparison under standardized conditions, future research should explicitly investigate multi-turn, interactive development scenarios to better capture the ecological validity of AI-assisted programming.

The focus on Python as the exclusive programming language limits conclusions about model performance across diverse development ecosystems. Different programming languages present unique syntactic, semantic, and paradigmatic challenges that could reveal alternative performance patterns. Languages with different complex characteristics,

such as functional programming languages or systems programming languages, might demonstrate different relative model capabilities.

Furthermore, the exclusive reliance on the HumanEval benchmark and Python constrains the broader generalizability of our results. While HumanEval provides a standardized and widely adopted evaluation framework, it remains limited to a narrow range of algorithmic tasks and cannot fully represent the breadth of real-world software engineering challenges, such as distributed systems, large-scale codebases, or multi-language development environments. Similarly, Python's high-level, dynamically typed syntax may bias performance toward certain model capabilities. Statically typed or lower-level languages (e.g., Java, C++, Rust) introduce different constraints related to type safety, memory management, and compilation that could lead to different relative performance outcomes. Therefore, while our findings are robust within the HumanEval Python context, caution should be exercised in generalizing them to other languages or practical enterprise scenarios. Future research should expand evaluation to multi-language benchmarks and heterogeneous datasets to assess whether the observed performance differences between Anthropic Claude and OpenAI GPT models persist under more diverse conditions.

Temporal factors also present validity concerns, as AI models continue evolving rapidly, potentially altering performance characteristics between evaluation time and practical deployment. The evaluation captured model performance at specific points in their development cycles, and continued training or architectural modifications could significantly change relative capabilities. Additionally, API response variability, while controlled through temperature settings, may introduce minor inconsistencies that could affect reproducibility of specific numerical results.

The findings align with recent studies demonstrating variable performance across different AI coding assistants, while extending understanding through comprehensive multi-dimensional analysis. Previous research has noted differences between OpenAI GPT and Anthropic Claude models in specific contexts, but this study provides the first systematic comparison across multiple code quality dimensions using standardized evaluation protocols. The positive correlation between complexity and correctness supplements existing literature that has primarily focused on binary success metrics without considering solution sophistication.

The superior performance of Anthropic Claude models corroborates industry reports indicating competitive advantages in reasoning-intensive tasks, while providing empirical evidence for these claims in programming contexts. However, the magnitude of performance differences observed exceeds those reported in some previous studies, potentially reflecting improvements in newer model versions or differences in evaluation methodologies. The consistency of Anthropic Claude advantages across multiple metrics strengthens confidence in these findings compared to studies examining single performance dimensions.

The code quality metrics employed in this study extend beyond functional correctness measures used in most existing evaluations, providing a more comprehensive assessment framework. While previous research has established the utility of the HumanEval benchmark for basic capability assessment, this study demonstrates the value of incorporating software engineering metrics for practical technology selection decisions. The maintainability and complexity analyses offer novel insights not captured in purely correctness-focused evaluations common in existing literature.

5. Conclusions

This comprehensive comparative analysis provides definitive evidence for substantial performance differences between AI model families in Python code generation tasks,

with Anthropic Claude models demonstrating consistent superiority across functional correctness, code sophistication, and maintainability metrics. Claude Sonnet 4 achieved the highest overall performance with a 95.1% success rate, establishing new benchmarks for AI coding assistant capabilities. The systematic evaluation of six state-of-the-art models across multiple dimensions reveals that sophisticated solutions incorporating appropriate complexity enhance rather than compromise software quality outcomes.

This research makes several significant contributions to the scientific understanding of AI code generation capabilities. First, it establishes the importance of multi-dimensional evaluation frameworks that extend beyond simple correctness metrics to encompass code quality characteristics crucial for practical software development. The positive correlation between cyclomatic complexity and functional correctness challenges conventional assumptions about optimal solution characteristics, suggesting that appropriate algorithmic sophistication enhances rather than hinders solution effectiveness.

Second, the study provides empirical evidence for systematic differences between AI model families in code generation approaches, demonstrating that constitutional AI principles and specialized training methodologies yield measurable advantages in programming tasks. The consistent pattern of Anthropic Claude model superiority across multiple metrics indicates fundamental architectural or training differences rather than random performance variations, contributing to theoretical understanding of effective AI system design for code generation applications.

Third, the comprehensive statistical analysis with effect size quantification establishes robust foundations for evidence-based technology selection in software development contexts. The magnitude of observed performance differences (over 20% in most metrics) provides clear practical significance that extends beyond statistical significance, enabling confident decision-making for organizations evaluating AI coding assistant integration.

For software development organizations, this research provides clear guidance for AI coding assistant selection based on specific project requirements and team capabilities. Organizations prioritizing functional correctness and solution robustness should strongly consider Anthropic Claude models, particularly Claude Sonnet 4 or Claude Opus 4, which demonstrated superior performance across all evaluation dimensions. The higher cyclomatic complexity of Anthropic-generated solutions requires teams with adequate code review capabilities but provides substantial benefits in solution reliability and completeness.

Development teams working on complex algorithmic challenges, system integration tasks, or applications requiring robust error handling will benefit most from Anthropic Claude models' sophisticated problem-solving approaches. However, teams with limited code review resources or junior developers may find OpenAI GPT models' simpler solutions more manageable, despite lower success rates. The trade-off between solution sophistication and code comprehensibility must be evaluated based on specific organizational contexts and developer skill levels.

Beyond general organizational guidance, the findings also hold distinct implications for different categories of users. For students and novice programmers, OpenAI GPT's simpler solutions may provide a gentler learning curve, producing shorter and more easily understood code that facilitates comprehension and foundational skill development. For professional developers, particularly those addressing complex algorithmic or system-level tasks, Anthropic Claude models' more sophisticated and reliable outputs can enhance productivity by reducing debugging time and improving solution robustness. For organizations with predominantly junior teams, the comprehensibility of OpenAI GPT generated solutions may help maintain codebase accessibility, whereas organizations with experienced teams are better equipped to manage the higher complexity of Anthropic Claude outputs through established code review and integration processes. This user-centered

perspective highlights that optimal AI coding assistant selection depends not only on absolute performance metrics but also on the expertise level and workflow context of the intended users.

Cost–benefit analysis should incorporate not only licensing expenses but also productivity gains from higher success rates, reduced debugging time from more robust solutions, and long-term maintenance implications of different code quality characteristics. Organizations should conduct pilot evaluations within their specific development contexts to validate these general findings against their requirements and constraints.

Several important research directions emerge from this study's findings and limitations. Comprehensive evaluation across multiple programming languages would establish the generalizability of observed performance patterns and identify language-specific model capabilities. Languages with different paradigmatic characteristics, such as functional programming or systems programming languages, may reveal alternative relative model strengths that could inform specialized application domains.

Investigation of real-world development workflows incorporating iterative refinement, human collaboration, and integration with existing codebases would provide crucial insights into practical AI-assisted development effectiveness. Static benchmark evaluation, while valuable for standardized comparison, cannot capture the dynamic aspects of human-AI collaboration that significantly impact development productivity and code quality outcomes in professional contexts.

Long-term maintainability studies tracking AI-generated code through multiple modification cycles would illuminate the practical implications of different solution characteristics over extended software evolution periods. Understanding how different code quality metrics predict actual maintenance effort and modification difficulty would refine guidance for optimal AI coding assistant selection and usage patterns.

Security and robustness evaluation represents another critical research direction, as AI-generated code increasingly appears in production systems. Systematic assessment of vulnerability patterns, error handling robustness, and security best practice adherence across different models would inform risk management strategies for AI-assisted development adoption.

Finally, investigation of human-AI collaboration patterns and optimal integration strategies would advance understanding of how to maximize the benefits of AI coding assistance while mitigating potential limitations. Research into developer training needs, code review protocols, and organizational adaptation strategies would support successful AI-assisted development implementation across diverse software development contexts.

The findings have significant implications for the rapidly evolving AI-assisted development tools market and software engineering practices more broadly. The substantial performance differences demonstrated between model families suggest that competitive advantages in AI coding assistance depend critically on underlying model capabilities rather than primarily on user interface design or integration features. This insight has strategic implications for both AI model developers and software development tool vendors.

For the broader software engineering community, this research provides evidence that AI coding assistants have matured sufficiently to serve as reliable development tools rather than experimental novelties. The high success rates achieved by leading models, particularly Claude Sonnet 4's 95.1% performance, indicate that AI-generated code can meet professional software development standards when appropriate models are selected and properly integrated into development workflows.

The implications extend to software engineering education and professional development, as the demonstrated effectiveness of AI coding assistants necessitates evolution in developer skill requirements and training programs. Future software engineers will likely

need expertise in AI collaboration, code review of AI-generated solutions, and strategic selection of AI tools based on project requirements rather than traditional implementation-focused skill sets alone.

This research establishes foundational evidence for informed decision-making in AI-assisted software development while identifying crucial areas for continued investigation. As AI coding assistants become increasingly central to software development workflows, comprehensive evaluation frameworks and evidence-based selection criteria become essential for realizing their potential benefits while managing associated risks and limitations.

Author Contributions: Conceptualization, A.B., G.G.M.D. and M.D.; methodology, A.B., G.G.M.D. and M.D.; software, A.B., G.G.M.D. and M.D.; validation, A.B., G.G.M.D. and M.D.; formal analysis, A.B., G.G.M.D. and M.D.; investigation, A.B., G.G.M.D. and M.D.; resources, A.B., G.G.M.D. and M.D.; data curation, A.B., G.G.M.D. and M.D.; writing—original draft preparation, A.B., G.G.M.D. and M.D.; writing—review and editing, A.B., G.G.M.D. and M.D.; visualization, A.B., G.G.M.D. and M.D.; supervision, A.B., G.G.M.D. and M.D. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data presented in this study is available on request from the corresponding author. The data is not publicly available due to privacy.

Acknowledgments: During the preparation of this manuscript, the author(s) used CahtGPT (OpenAI) GPT-5 for the purpose of enhancing language and readability. The authors have reviewed and edited the output and take full responsibility for the content of this publication.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; de Oliveira Pinto, H.P.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. Evaluating Large Language Models Trained on Code. Available online: https://arxiv.org/pdf/2107.03374 (accessed on 14 July 2025).
2. Hou, X.; Zhao, Y.; Liu, Y.; Yang, Z.; Wang, K.; Li, L.; Luo, X.; Lo, D.; Grundy, J.; Wang, H. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Trans. Softw. Eng. Methodol.* **2024**, *33*, 1–79. [CrossRef]
3. Ko, E.; Kang, P. Evaluating Coding Proficiency of Large Language Models: An Investigation Through Machine Learning Problems. *IEEE Access* **2025**, *13*, 52925–52938. [CrossRef]
4. Russo, D. Navigating the Complexity of Generative AI Adoption in Software Engineering. *ACM Trans. Softw. Eng. Methodol.* **2024**, *33*, 1–50. [CrossRef]
5. Haindl, P.; Weinberger, G. Does ChatGPT help novice programmers write better code? Results from static code analysis. *IEEE Access* **2024**, *12*, 114146–114156. [CrossRef]
6. Alenezi, M.; Akour, M. Ai-driven innovations in software engineering: A review of current practices and future directions. *Appl. Sci.* **2025**, *15*, 1344. [CrossRef]
7. Sauvola, J.; Tarkoma, S.; Klemettinen, M.; Riekki, J.; Doermann, D. Future of software development with generative AI. *Autom. Softw. Eng.* **2024**, *31*, 1–8. [CrossRef]
8. Niu, C.; Zhang, T.; Li, C.; Luo, B.; Ng, V. On Evaluating the Efficiency of Source Code Generated by LLMs. In Proceedings of the 2024 IEEE/ACM 1st International Conference on AI Foundation Models and Software Engineering, FORGE 2024, New York, NY, USA, 14 April 2024.
9. Nguyen-Duc, A.; Cabrero-Daniel, B.; Przybylek, A.; Arora, C.; Khanna, D.; Herda, T.; Rafiq, U.; Melegati, J.; Guerra, E.; Kemell, K.-K.; et al. Generative artificial intelligence for software engineering—A research agenda. *Softw. Pract. Exp.* 2025; *in press*. [CrossRef]
10. Palla, D.; Slaby, A. Evaluation of Generative AI Models in Python Code Generation: A Comparative Study. *IEEE Access* **2025**, *13*, 65334–65347. [CrossRef]

11. Idrisov, B.; Schlippe, T. Program Code Generation with Generative Ais. *Algorithms* **2024**, *17*, 62. [CrossRef]
12. Liu, Y.; Le-Cong, T.; Widyasari, R.; Tantithamthavorn, C.; Li, L.; Le, X.B.D.; Lo, D. Refining chatgpt-generated code: Characterizing and mitigating code quality issues. *ACM Trans. Softw. Eng. Methodol.* **2024**, *33*, 1–26. [CrossRef]
13. Ouyang, S.; Zhang, J.M.; Harman, M.; Wang, M. An empirical study of the non-determinism of chatgpt in code generation. *ACM Trans. Softw. Eng. Methodol.* **2025**, *34*, 1–28. [CrossRef]
14. Almanasra, S.; Suwais, K. Analysis of ChatGPT-Generated Codes Across Multiple Programming Languages. *IEEE Access* **2025**, *13*, 23580–23596. [CrossRef]
15. Kalyan, K.S. A survey of GPT-3 family large language models including ChatGPT and GPT-4. *Nat. Lang. Process. J.* **2024**, *6*, 100048. [CrossRef]
16. Gu, X.; Chen, M.; Lin, Y.; Hu, Y.; Zhang, H.; Wan, C.; Wei, Z.; Xu, Y.; Wang, J. On the effectiveness of large language models in domain-specific code generation. *ACM Trans. Softw. Eng. Methodol.* **2025**, *34*, 1–22. [CrossRef]
17. Zheng, Z.; Ning, K.; Wang, Y.; Zhang, J.; Zheng, D.; Ye, M.; Chen, J. A survey of large language models for code: Evolution, benchmarking, and future trends. *arXiv* **2023**, arXiv:2311.10372.
18. Pasquale, L.; Sabetta, A.; d'Amorim, M.; Hegedűs, P.; Mirakhorli, M.T.; Okhravi, H. Challenges to Using Large Language Models in Code Generation and Repair. *IEEE Secur. Priv.* **2025**, *23*, 81–88. [CrossRef]
19. Liu, M.; Wang, J.; Lin, T.; Ma, Q.; Fang, Z.; Wu, Y. An Empirical Study of the Code Generation of Safety-Critical Software Using LLMs. *Appl. Sci.* **2024**, *14*, 1046. [CrossRef]
20. Liu, Z.; Tang, Y.; Luo, X.; Zhou, Y.; Zhang, L.F. No Need to Lift a Finger Anymore? Assessing the Quality of Code Generation by ChatGPT. *IEEE Trans. Softw. Eng.* **2024**, *50*, 1548–1584. [CrossRef]
21. Fawareh, H.; Al-Shdaifat, H.M.; Al-Refai, M.; Fawareh, F.A.N.; Khouj, M. Investigates the Impact of AI-generated Code Tools on Software Readability Code Quality Factor. In Proceedings of the 25th International Arab Conference on Information Technology, ACIT 2024, Zarqa, Jordan, 10–12 December 2024.
22. Tosi, D. Studying the Quality of Source Code Generated by Different AI Generative Engines: An Empirical Evaluation. *Future Internet* **2024**, *16*, 188. [CrossRef]
23. Chang, Y.; Wang, X.; Wang, J.; Wu, Y.; Yang, L.; Zhu, K.; Chen, H.; Yi, X.; Wang, C.; Wang, Y.; et al. A survey on evaluation of large language models. *ACM Trans. Intell. Syst. Technol.* **2024**, *15*, 1–45. [CrossRef]
24. Dong, Y.; Jiang, X.; Jin, Z.; Li, G. Self-Collaboration Code Generation via ChatGPT. *ACM Trans. Softw. Eng. Methodol.* **2024**, *33*, 1–38. [CrossRef]
25. Jain, R.; Thanvi, I.; Subasinghe, A. The evolution of ChatGPT for programming: A comparative study. *Eng. Res. Express* **2025**, *7*, 015242. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.

Reproduced with permission of copyright owner. Further reproduction prohibited without permission.